CAMPUS CAFE ORDERING SYSTEM

Java OOP Laboratory Exercise

Prepared by:

Dazzle Mc Kenzie Adaza

Anjo Gella

Section: E

Date: 09/12/2026

1. Introduction

The Campus Cafe Ordering System is a Java console program that records a customer's order of up to three menu items. It supports beverages, pastries, and meals. Each item type calculates its price using its own rules. Student customers receive a ten-percent discount, and customers earn one loyalty point for every 50 pesos spent.

2. Object-Oriented Programming Concepts

Classes and Objects

The program uses seven classes: MenuItem, Beverage, Pastry, Meal, Customer, Order, and CafeApp. CafeApp creates customer, menu-item, and order objects and makes them work together.

Abstraction

MenuItem is an abstract class. It stores common item information and declares computePrice() as an abstract method. Each child class supplies its own price calculation.

Inheritance

Beverage, Pastry, and Meal extend MenuItem. They reuse its name, base price, and item code instead of duplicating those common fields.

Encapsulation

Attributes are private or protected rather than public. Getter and setter methods control access. Setters reject invalid values such as negative prices, negative extra sides, blank names, and invalid beverage sizes.

Polymorphism

Order stores menu items using the MenuItem type. When computeSubtotal() calls computePrice(), Java runs the correct overridden method for each actual item type. The subclasses also override getDescription().

Constructors and Overloading

The classes provide default and parameterized constructors where required. Customer has three constructors. Customer.addPoints() and Order.computeTotal() are overloaded to support different input types and calculations.

Conditional Statements

if and if-else statements validate data and calculate beverage sizes, warmed pastry fees, meal options, student discounts, voucher totals, and the three-item order limit.

3. **Program Features**

- Supports up to three menu items per order.
- Calculates beverage prices for small, medium, and large sizes.
- Adds a warming fee to pastries when selected.
- Adds upsizing and extra-side fees to meals.
- Applies a ten-percent student discount.
- Supports voucher deductions without allowing a negative total.
- Awards one loyalty point for every 50 pesos spent.
- Prints receipts and demonstrates rejection of a fourth item and invalid negative price.

4. **Challenges and Solutions**

One challenge was designing the order so it could hold menu items of different subclasses without using an ArrayList or another collection. This was solved by creating three separate MenuItem fields. Another challenge was applying different price rules while keeping one common item type. The abstract computePrice() method and method overriding solve this through polymorphism. Input validation is handled in setter methods with conditional statements.

5. **Testing**

The CafeApp main method creates two customers and demonstrates beverages, pastries, and meals. It prints two receipts, tests the three-item limit by attempting to add a fourth item, applies a voucher to the second order, and attempts to set a negative base price to demonstrate validation.

6. **Conclusion**

The project demonstrates how Java OOP concepts can be combined to create a small ordering system. The use of abstraction, inheritance, encapsulation, polymorphism, constructors, and conditional statements makes the program organized and easier to maintain.

Note: Run the Java program and attach screenshots of its actual console output before submission if your instructor requires output screenshots.